

Lecture 3: OS Overview

Professor G. Sandoval

03



Some slides adapted by G. Sandoval for CS6233, from Tanenbaum & Bo, Modern Operating Systems: 4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved. Also from Slides by Brendan Dolan-Gavitt

Today's questions

- Which work needs the kernel?
- How do arguments and file descriptors reach a program?
- How do protection boundaries affect failures and access?

User mode and kernel mode

- The kernel is the privileged core of the operating system.
- Ordinary programs run in user mode with limited access.
- They request kernel services through system calls.
- The kernel checks requests for resources such as files and devices.

User/Kernel/System Calls

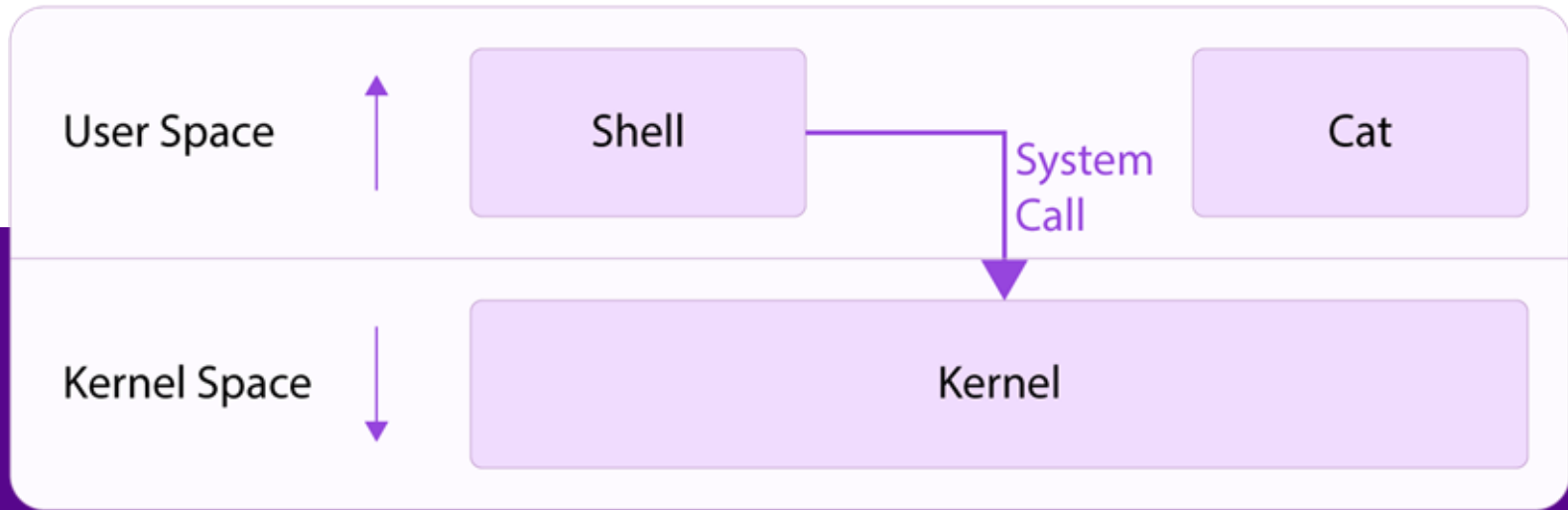


Figure 0-1. A kernel and two user processes.

Which work needs the kernel?

Add two integers already in registers

0%

Count newlines in an array already in memory

0%

Send bytes to the terminal

0%

Open a file by its pathname

0%

C and D

0%

Answer: C and D need the kernel

- A and B can run entirely in user mode.
- C asks the kernel to send bytes to the terminal.
- D asks the kernel to find the file and check access.
- A C library function can do user-space work before a syscall.

System calls by purpose

- Processes: fork, exec, wait, exit
- Byte I/O: open, read, write, close
- Connections: pipe, dup
- Names and metadata: chdir, mkdir, link, unlink, fstat



NYU

XV6



xv6 as a teaching OS

- MIT's small Unix-like OS models Unix Version 6.
- We can follow a request from a user program into the kernel.
- These examples use the classic x86 xv6 interface.
- Host-shell examples and Unix permissions are labeled separately.

QEMU: host and guest

- The host runs QEMU as a program.
- The guest runs xv6 on the machine QEMU supplies.
- A guest crash normally leaves the host usable.
- We can inspect and restart the guest.

Demo: tracing I/O

In the xv6 shell:

```
wc README
```

```
wc < README
```

```
cat README | wc
```

Predict each input and output before the demo.

Command-line arguments

```
int main(int argc, char *argv[])
```

argc counts the argument strings.

argv contains pointers to those strings.

argv[argc] is a null pointer.

What does wc receive?

wc README

argc = ?

argv[0] = ?

argv[1] = ?

argv[2] = ?

Solo 45 sec. Compare before the reveal.

Answer: two arguments

```
argc = 2
```

```
argv[0] = "wc"
```

```
argv[1] = "README"
```

```
argv[2] = 0 // null pointer
```

argv[argc] marks the end of the pointer list.

The xv6 user interface

```
#include "types.h"  
#include "user.h"  
#include "fcntl.h"  
  
printf(1, "hello\n"); // xv6 x86  
printf(2, "error\n");
```

File descriptors

- A descriptor is a small integer in one process.
- The kernel uses it to find an open I/O object.
- Files, pipes, and devices share read/write operations.
- The same number in two processes can name different objects.

The standard descriptors

0 Standard input

1 Standard output

2 Standard error

- A shell normally sets these up before running a command.

Which descriptor?

- A `printf(1, "hello\n");`
- B `read(0, buf, sizeof buf);`
- C `printf(2, "error\n");`

Pairs 60 sec. Name the descriptor and direction.

Answer: 1 out, 0 in, 2 out

- A sends formatted output through descriptor 1.
- B receives bytes through descriptor 0.
- C sends a diagnostic through descriptor 2.
- The shell can choose a different destination for each output.

Opening a file in xv6

```
int fd = open("README", O_RDONLY);
```

Success: a nonnegative descriptor.

Failure: -1. Check before using fd.

open chooses the lowest unused descriptor number.

Which number will open return?

Initially, only 0, 1, and 2 are open. close and open succeed.

```
close(1);
```

```
int fd = open("README", O_RDONLY);
```

What is fd? What happens to write(1, ...)?

Solo 45 sec. Then compare descriptor tables.

Answer: descriptor 1

- `close(1)` leaves the lowest available slot at 1.
- `open` puts the read-only README file in that slot.
- `fd` is 1, and `write(1, ...)` fails because this open file is read-only.
- The number does not permanently mean “the screen.”

read returns a byte count

```
int n = read(fd, buf, sizeof buf);
```

$n > 0$ Process exactly n bytes.

$n == 0$ End of input for this stream.

$n < 0$ Handle the error.

The buffer does not automatically become a C string.

How many bytes should we write?

```
char buf[8];  
int n = read(0, buf, sizeof buf);  
// n == 3, bytes are: A, \0, B  
write(1, buf, sizeof buf);
```

What is wrong? Would `strlen(buf)` fix it?

Pairs 2 min. Fix the length and defend it.

Answer: write the bytes read

`write(1, buf, n); // n is 3 here`

`sizeof buf` is capacity, not the received length.

`strlen` stops at the embedded null byte.

Use the returned count, then check the write result.

I/O redirection

```
grep the < README > found.txt
```

Descriptor 0 reads from README.

Descriptor 1 writes to found.txt.

Descriptor 2 keeps its previous destination.

Which words reach grep?

grep the < README > found.txt

Does grep receive six strings, four strings, or two?

Where do README and found.txt go instead?

Pairs 90 sec. Write argv and the two descriptors.

Answer: two arguments

`argc = 2`

`argv = {"grep", "the", 0}`

Descriptor 0: README

Descriptor 1: found.txt

The shell handles `<` and `>` before `grep` runs.

Pipes

- A pipe has a read end and a write end.
- The shell connects one program's output to another's input.
- The kernel buffers bytes between them.
- After the buffer drains, EOF requires all write ends to close.

trace wc's input and output: cat < README | wc > counts.txt

README → terminal

0%

Pipe → terminal

0%

Pipe → counts.txt

0%

README → counts.txt

0%

None of the above

0%

Answer: the pipeline

- cat reads README through descriptor 0.
- cat writes to the pipe through descriptor 1.
- wc reads the pipe through descriptor 0.
- wc writes counts.txt through descriptor 1.
- Both diagnostics still use descriptor 2 at the terminal.

Where do hello and error go?

Host C and host shell:

```
printf("hello\n");  
fprintf(stderr, "error\n");
```

./a.out > out.txt

hello: ? error: ?

Vote, then explain which descriptor changed.

Answer: only stdout moved

- hello goes to out.txt through descriptor 1.
- error still goes to the terminal through descriptor 2.
- `> out.txt` changes stdout.
- Keeping diagnostics separate lets pipelines process ordinary output.

Does redirection order matter?

A `./a.out > out.txt 2>&1`

B `./a.out 2>&1 > out.txt`

For each case, where does error go?

Optional host-shell challenge. Trace left to right.

Answer: redirection order

A: hello and error both go to out.txt.

B: hello goes to out.txt, error stays at the terminal.

2>&1

Descriptor 2 takes descriptor 1's target at that point.

A copy loop

```
char buf[512];
int n;
while ((n = read(0, buf, sizeof buf)) > 0) {
    if (write(1, buf, n) != n) {
        printf(2, "write error\n");
        exit();
    }
}
if (n < 0) printf(2, "read error\n");
```

Who does each part?

- 1 Recognizes | and > in the command line
- 2 Counts words in incoming bytes
- 3 Maintains the pipe and transfers bytes for read/write
- 4 Uses descriptor 0 without needing the input pathname

Name the shell, utility, or kernel. Explain one choice.

Where should a driver run?

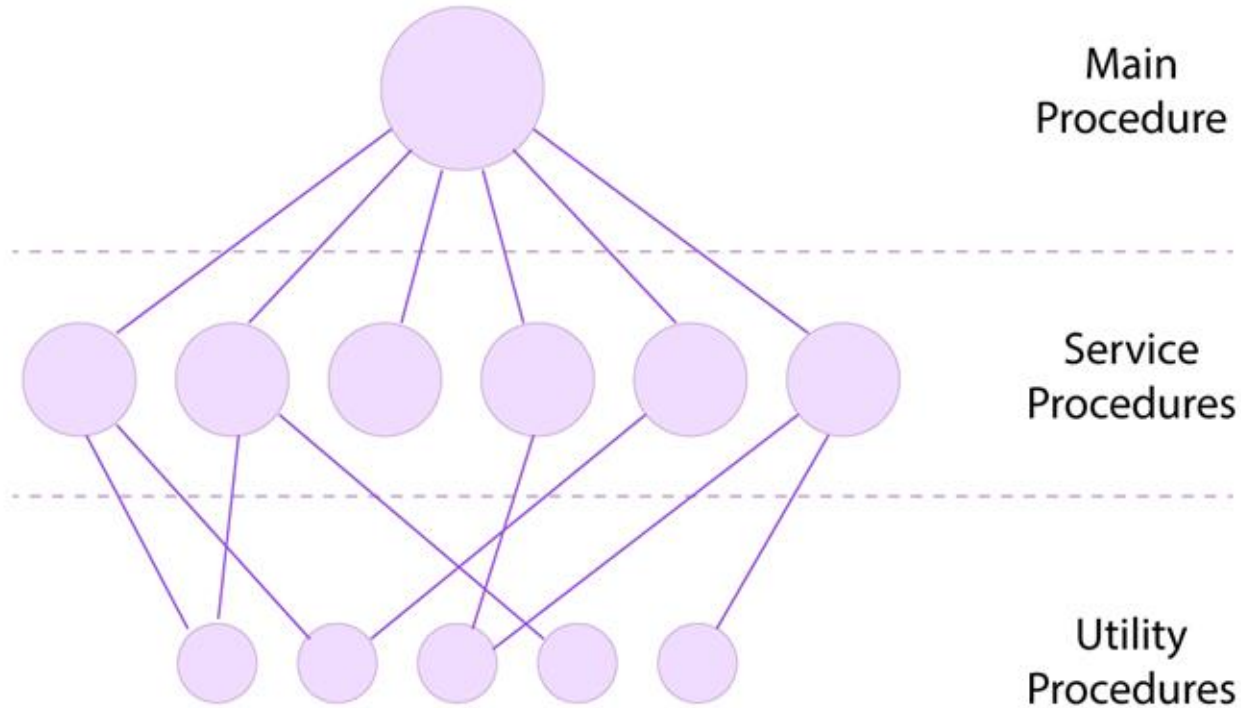
- A driver makes a bad pointer write.
- Design A: the driver runs inside the kernel.
- Design B: it runs in a separate user process.
- Which contains the damage? What might it cost?

Pairs 60 sec. Predict the damage and the tradeoff.

Monolithic kernels

- Most OS services execute in one kernel address space.
- Components can call each other directly.
- A bad pointer in a driver can corrupt kernel state.
- xv6 and Linux are examples of this organization.

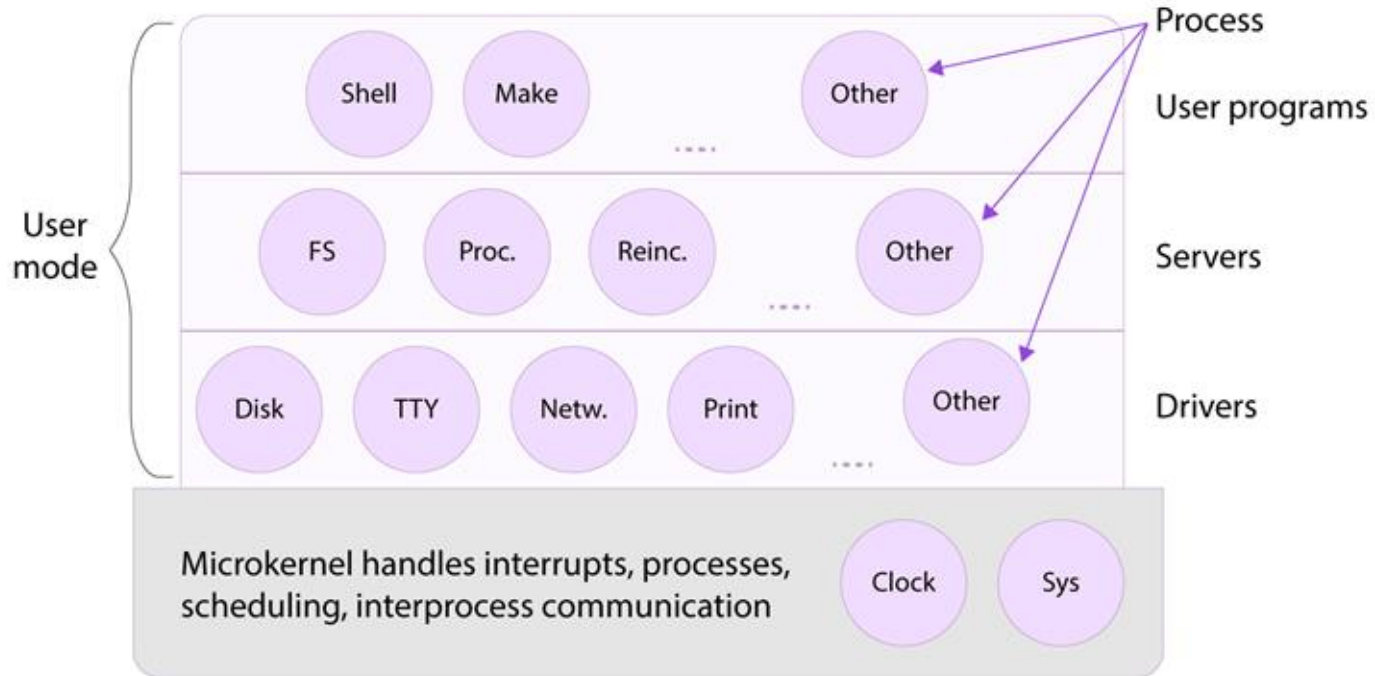
Monolithic Organization



Microkernels

- A small kernel supplies core protection and communication.
- Many OS services run as separate user-space servers.
- Requests can cross process boundaries through messages.
- An isolated service may restart without restarting the kernel.

Microkernel Organization



Boundary costs

- Messages may add copying and scheduling work.
- Direct calls avoid some of that overhead.
- Separate services can improve fault isolation.
- The result depends on the implementation and workload.

What survives a driver crash?

- The printer service fails during a print job.

A It shares the kernel's address space.

B It runs as a separate user-space service.

- For each design, name what might fail and what could restart.

Pairs 2 min. Give one benefit and one limitation.

Answer: fault isolation

- A driver in the kernel can damage shared kernel state.
- A protected service process cannot normally overwrite kernel memory.
- Restarting that service still requires restoring useful state.
- The application may need to retry its failed request.



General OS Abstractions

A process is a running program

- Two runs of the same executable create distinct processes.
- Each has execution state and a virtual address space.
- The OS saves and restores CPU state when switching execution.
- A process also has descriptors and other kernel-managed state.

Address spaces isolate memory

- User programs work with virtual addresses.
- Each process has its own address-space mappings.
- Hardware checks mappings and permissions set by the OS.
- Explicit shared mappings can allow controlled sharing.

Same address, same memory?

- Process A prints a pointer: 0x4000
- Process B prints a pointer: 0x4000
- A writes through its pointer. Must B see the change?
- Assume ordinary private writable mappings, with no shared memory.

Commit to yes or no. Explain what the number names.

Answer: separate address spaces

- The same virtual number can map to different physical memory.
- A's write changes A's private object.
- B's mapping need not refer to that object.
- Sharing requires an explicit shared mapping or another communication path.

Files and descriptors

- A regular file contains bytes and associated metadata.
- A pathname lets a process request access with open.
- A descriptor refers to an open object within that process.
- Later read/write calls use the descriptor rather than the pathname.

One I/O interface

- A read can receive bytes from a file, terminal, or pipe.
- A write can send bytes to a file, terminal, or pipe.
- The object still determines details such as blocking and seeking.
- Descriptor 1 tells us the slot, not the kind of object.

Users and groups

- A Unix file has an owner and a group.
- A process carries a user identity and group memberships.
- Permissions select which identities may access the file.
- This section describes Unix/Linux, beyond stock xv6.

Unix permission bits

-rw-r----- alice staff report.txt

Owner: rw- Read and write

Group: r-- Read only

Others: --- No access

r = read w = write x = execute - = absent

One permission class applies

- If the process is the file's owner, use the owner bits.
- Otherwise, if a group membership matches, use the group bits.
- Otherwise, use the other bits.
- Do not combine classes or fall through after a denial.

Who can read the file?

```
--w-r--r--  alice  staff  report.txt
```

Alice owns the file and belongs to staff.

Bob belongs to staff. Cara does not.

Who may read it now? Assume no ACLs or special privileges.

Pairs 2 min. Choose a class before deciding access.

Answer: Bob and Cara can read

- Alice uses owner bits: -w-. Read is denied.
- Bob uses group bits: r--. Read is allowed.
- Cara uses other bits: r--. Read is allowed.
- Alice's staff membership does not override the owner match.

What changes after chmod?

```
-rw-r--r--  alice  staff  report.txt
```

```
chmod g+w,o-r report.txt
```

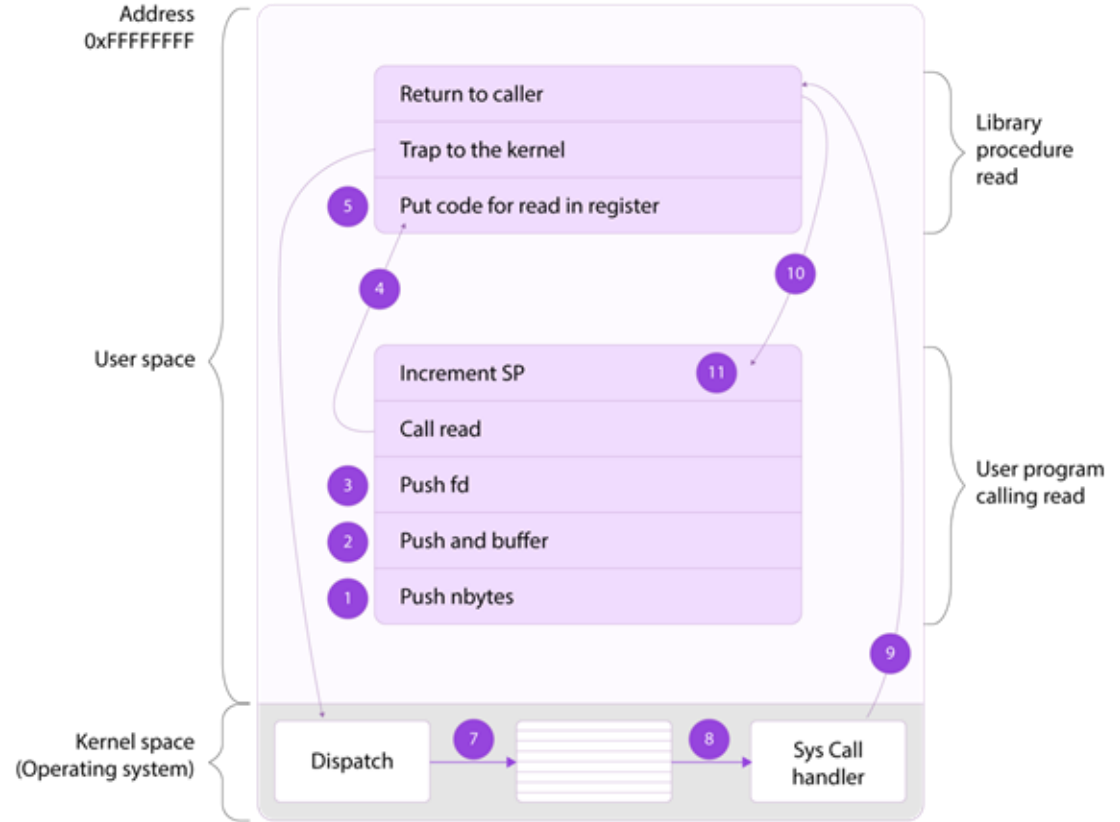
What is the new mode? Which classes changed?

Solo 60 sec. Write all nine permission bits.

The system-call boundary

- C code calls a library wrapper such as read.
- The wrapper follows the machine's syscall convention.
- A trap or syscall instruction enters the kernel.
- The kernel validates arguments, performs the request, and returns.

System Calls



Historical 32-bit example of read
Which steps cross the privilege boundary?

Exit Ticket: Who connects the following pipe: `cat < README | wc`

The kernel parses `|` and `<`.

0%

The shell connects descriptors using kernel services.

0%

`wc` must open `README` by name.

0%

`cat` copies directly into `wc`'s private memory.

0%